LLM Basics

Yilong Wu

2026/01/24

Saint Mary's University

Large Language Models (LLMs)

Definition

An **LLM** is typically a Transformer-based neural language model trained on large text corpora to learn a conditional token distribution

It generates text and code by predicting sequences of tokens (words/subwords) based on patterns learned during training.

The weather today is _____ ?

01 Timeline

Timeline Overview

I

Foundations
(≈1950 - 2002)

II

Representations
(≈2003 - 2017)

III

Scale & Alignment
(≈2018 - 2024)

IV

Agents
(≈2024 – 2025 +)

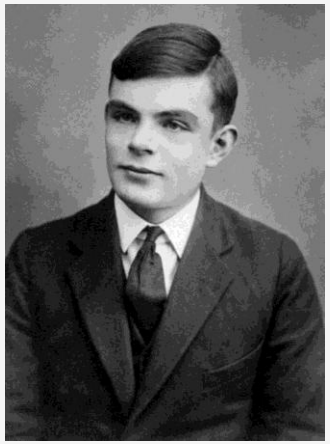
Turing

Intelligence as Observable Behavior

Turing proposed that if a machine can engage in conversation indistinguishably from a human, we should consider it intelligent.
This pragmatic definition shifts focus from "thinking" to observable performance.

Language Becomes a Testing Ground

The Turing Test established language as the ultimate benchmark for AI.
Seven decades later, LLMs are essentially solving the Turing Test at scale.



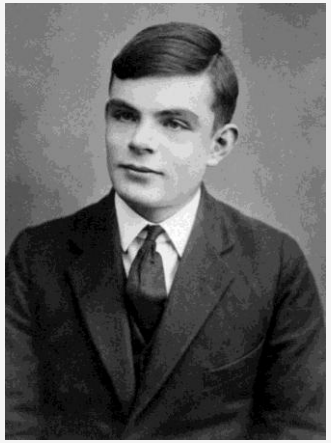
Alan Turing (1912 – 1954)

The Question

"Can machines think?"

Turing's 1950 paper "Computing Machinery and Intelligence" reframed the philosophical question into an operational test.

Turing

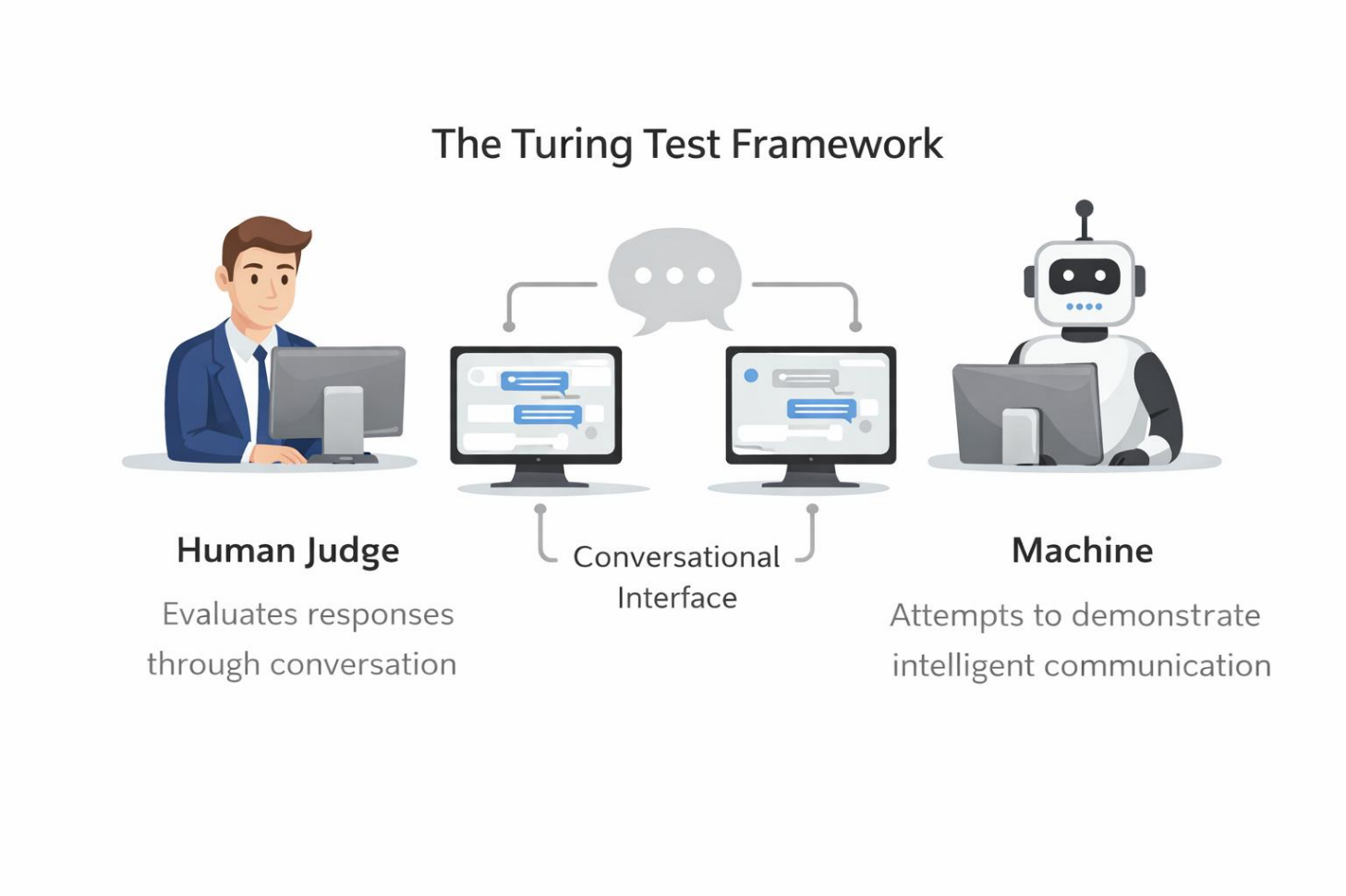


Alan Turing (1912 – 1954)

The Question

"Can machines think?"

Turing's 1950 paper "Computing Machinery and Intelligence" reframed the philosophical question into an operational test.



Neural Language Models

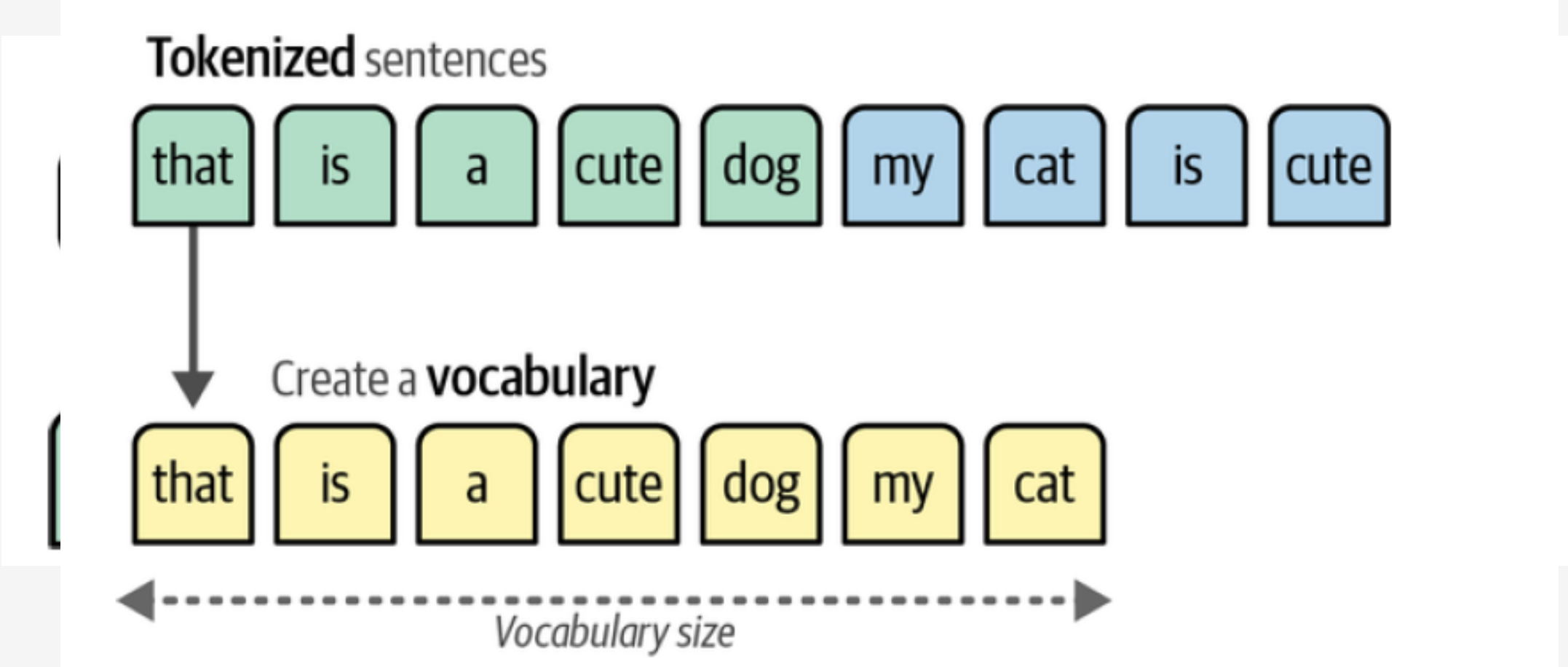
Neural Networks Model Sequence Probability

Bengio et al. demonstrated that neural networks can learn to predict the next word in a sequence by learning distributed representations.
This was the first practical step toward modern language models.

Early "Embeddings + Language Modeling" Foundation

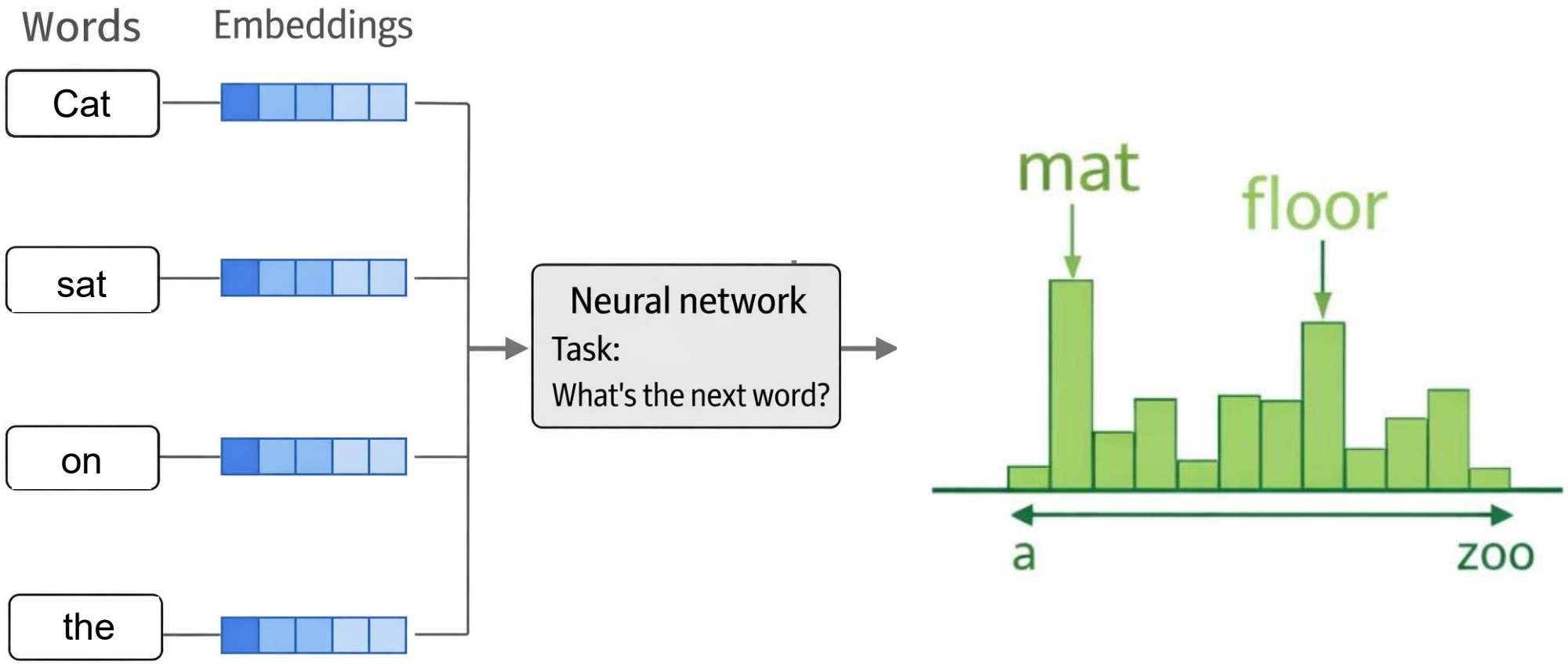
This work introduced the idea that words could be represented as dense vectors (embeddings) where similar words have similar vectors—making semantic computation possible.

Bag-of-Words (BoW)



Alammar and Grootendorst (2024)

Neural Language Models



Word Embeddings Go Mainstream

Vector Representations Make Meaning Computable at Scale

Word2Vec (Mikolov et al.) showed that simple neural networks can learn high-quality word vectors that capture semantic relationships.
Suddenly, we could do math with meaning: king - man + woman ≈ queen.

NLP Adoption Accelerates

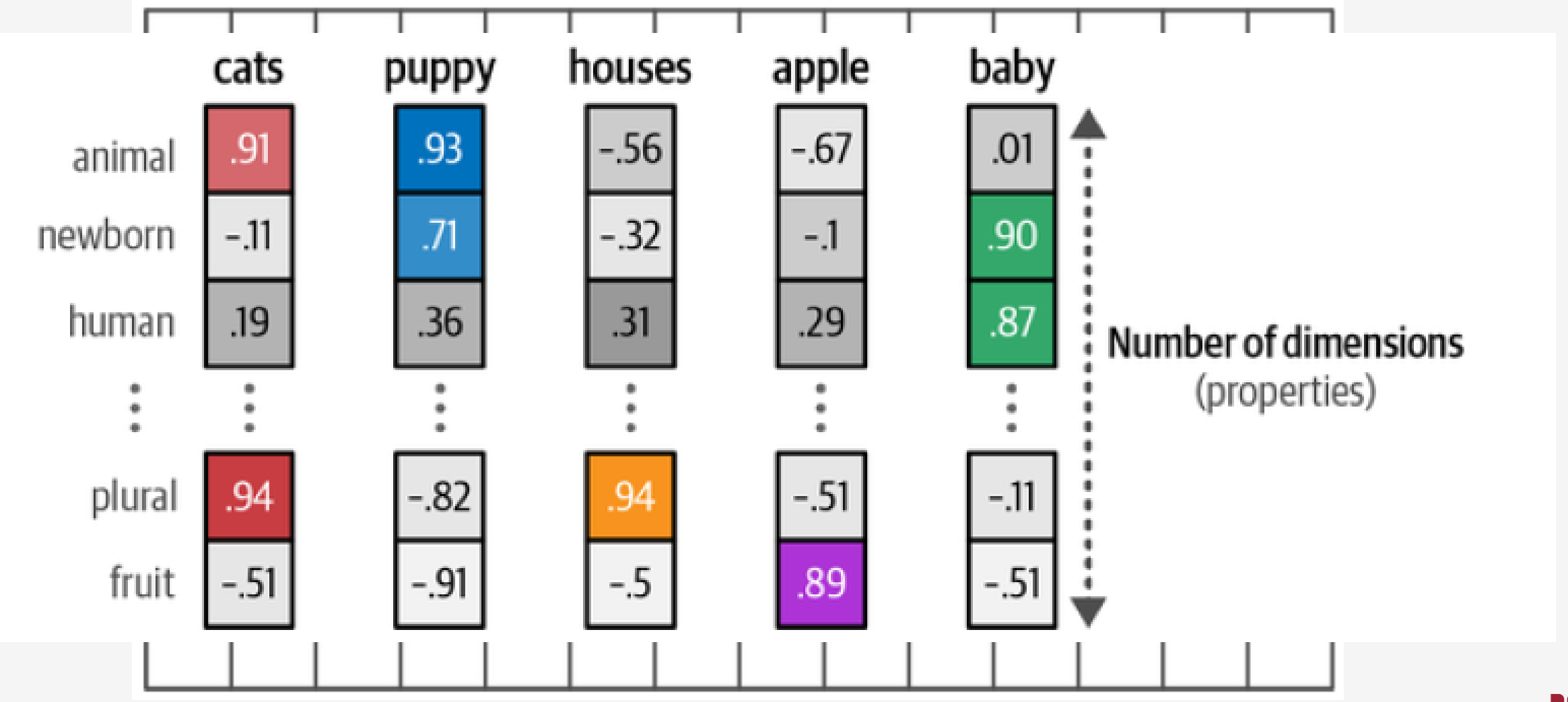
Pre-trained word embeddings became the standard foundation for virtually all NLP tasks. Instead of training from scratch, everyone could use these pre-trained vectors as a starting point.

Why It Matters

This is why "semantic similarity" became practical.

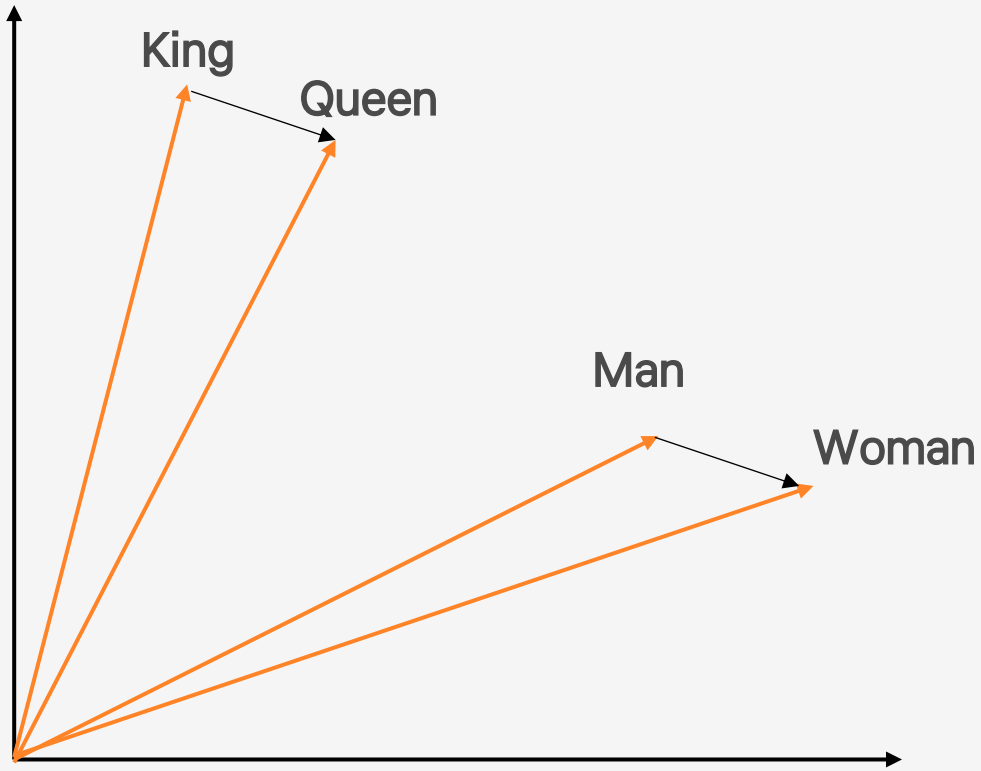
Word embeddings made it possible to measure meaning computationally, enabling search, recommendation, and clustering based on semantic similarity rather than just keywords.

Word Embeddings Go Mainstream



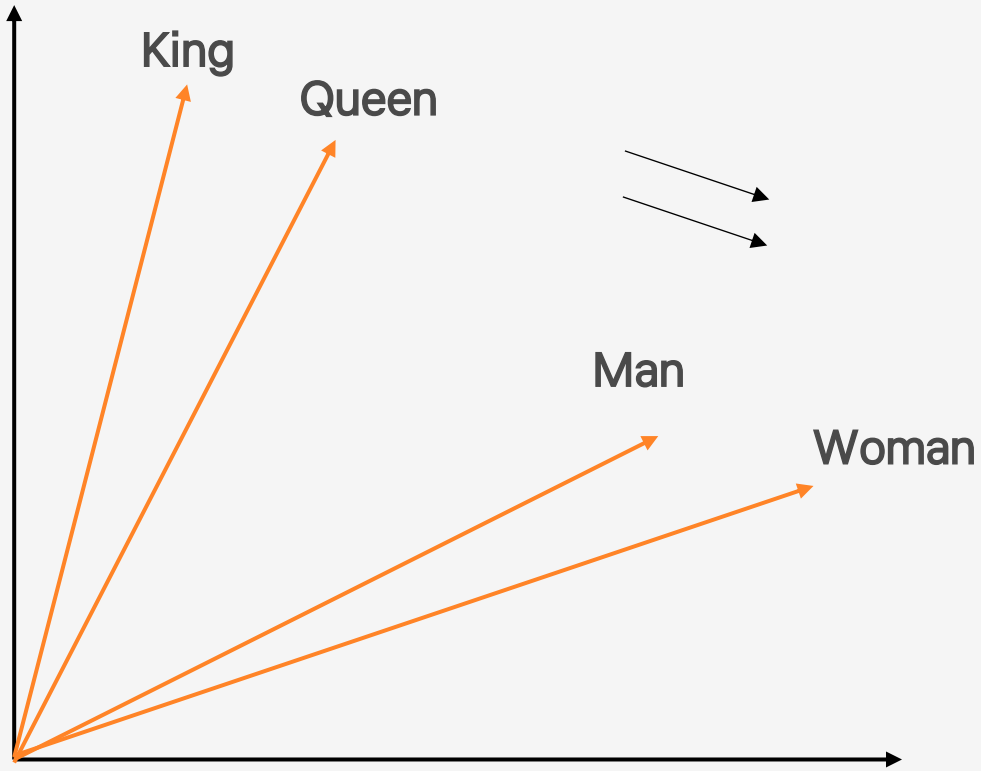
Alammar and Grootendorst (2024)

Word Embeddings Go Mainstream



Alammar and Grootendorst (2024)

Word Embeddings Go Mainstream



Alammar and Grootendorst (2024)

Attention

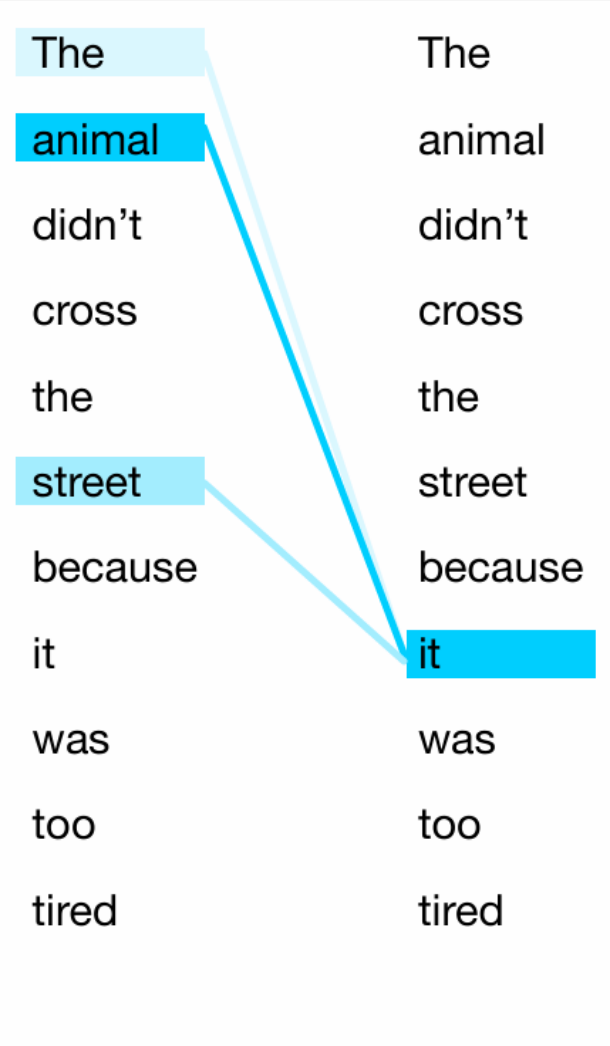
Dynamic Focus: "What Should I Look at Right Now?"

Bahdanau et al. introduced attention mechanisms to Seq2Seq models. Instead of compressing all input information into a single fixed vector, the decoder can "look at" different parts of the input at each step.

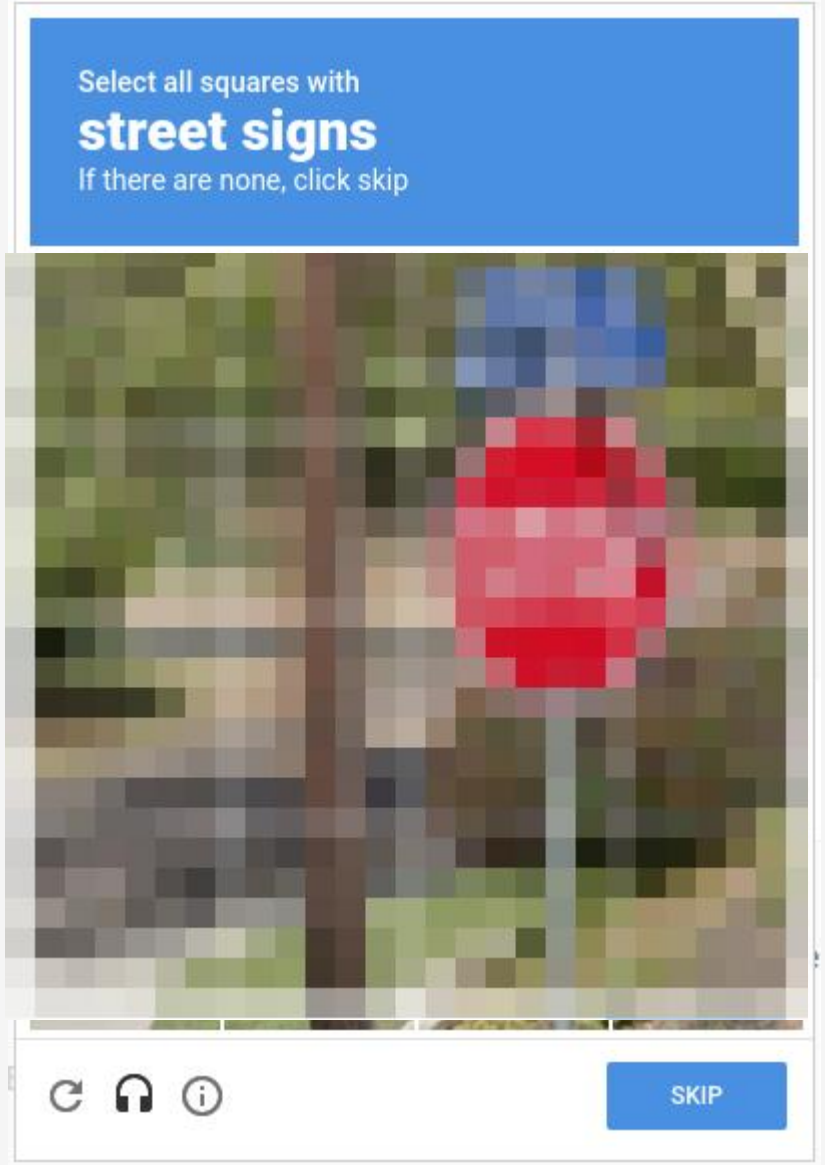
Attention Weights Visualization

	The	cat	sat	down
The	0.15	0.55	0.20	0.10
cat	0.10	0.45	0.35	0.10
sat	0.05	0.35	0.40	0.25
down	0.05	0.20	0.45	0.30

Darker cells indicate higher attention weights



Attention



Better Handling of Longer Inputs

Attention mechanisms solved the information bottleneck problem.
Models could now handle much longer sequences without losing information, dramatically improving translation quality.

Subword Tokenization

Open Vocabulary via Subword Units

Subword tokenization solves the out-of-vocabulary problem by breaking rare words into meaningful pieces.

Subword Tokenization Examples

Word-level	Character-level	Subword (BPE)	Common words
unfamiliarish	unfamiliarish	unfamiliarish	the
↓	↓	↓	↓
[UNK]	u n f a m i l i a r I s h	un familiar ish	the
Lost information	Too granular	Optimal balance	Preserved

Tokens: Why They Matter

Models Read/Write T "Words"

LLMs don't see words—they see tokens. "ChatGPT" might be 2 tokens, "prompting tokens depending on the tokenizer.

https://platform.openai.com/tokenize

BERT base model (uncased)	[CLS] english and capital ##ization [UNK] [UNK] sh ow _ token ##s false none el1 ##f = = > = else : two tab ##s : " " three tab ##s : " " 12 . 0 * 50 = 600 [SEP]
BERT base model (cased)	[CLS] English and CA ##PI ##TA ##L ##I ##Z ##AT ##I ON [UNK] [UNK] show _ token ##s F ##als ##e None el ##if = = > = else : two ta ##bs : " " Three ta ##bs : " " 12 . 0 * 50 = 600 [SEP]
GPT-2	English and CAP ITAL IZ ATION show _ t ok ens False None el if == >= else : two tab s :" " Three tabs : " " 12 . 0 * 50 = 600
FLAN-T5	English and CA PI TAL IZ ATION <unk> <unk> show _ to ken s Fal s e None e l if = = > = else : two tab s : " " Three tab s : " " 12. 0 * 50 = 600 </s>
GPT-4	English and CAPITAL IZATION show _tokens False None elif == >= else : two tabs :" " Three tabs : " " 12 . 0 * 50 = 600
StarCoder	English and CAPITAL IZATION show _ tokens False None elif == >= else : two tabs :" " Three tabs : " "

oken Calculation Examples

-4 chars/token

n fox jumps"

brown | fox | jumps

-3 chars/token

te() { return; }"

(|) | { | return | ; | }

00 words = ~3,000 pages

Tokens: Why They Matter

Token Count Drivers

Cost

APIs price by token, so token count directly impacts cost.

Latency

Longer contexts require more computation, leading to slower responses.

Context Limits

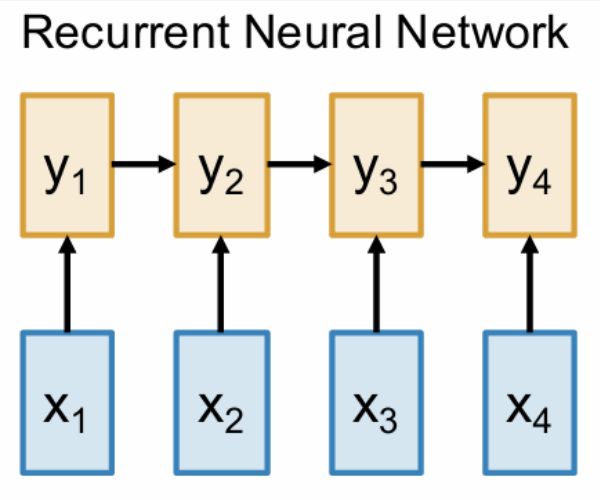
Context windows are measured in tokens, defining the amount of information the LLM can process.

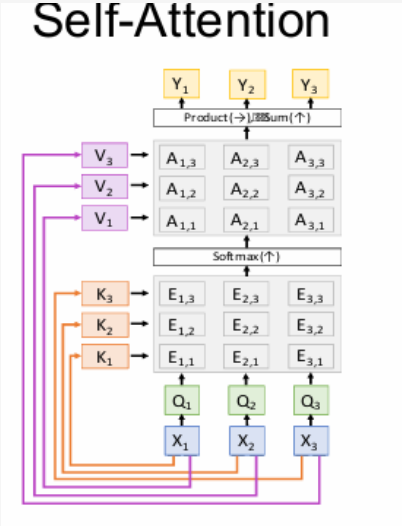
Transformer

Self-Attention Becomes the Core Primitive

Vaswani et al.'s "Attention Is All You Need" replaced recurrence entirely with self-attention.

Each position can attend to all positions in the sequence, enabling direct long-range dependencies.





https://bbycroft.net/llm

Transformer Intuition: Why Long Context Gets Expensive

This is why "just paste everything" is often a bad strategy

	The	cat	sat	down
The	0.15	0.55	0.20	0.10
cat	0.10	0.45	0.35	0.10
sat	0.05	0.35	0.40	0.25
down	0.05	0.20	0.45	0.30

Scaling + GPT-3 (Prompt as Interface)

In-Context Learning Paradigms

Zero-Shot

Instruction: "Translate to French: Hello world"

Output: "Bonjour le monde"

Few-Shot

Example: "Hello → Bonjour"

Query: "Goodbye → ?"

Output: "Au revoir"

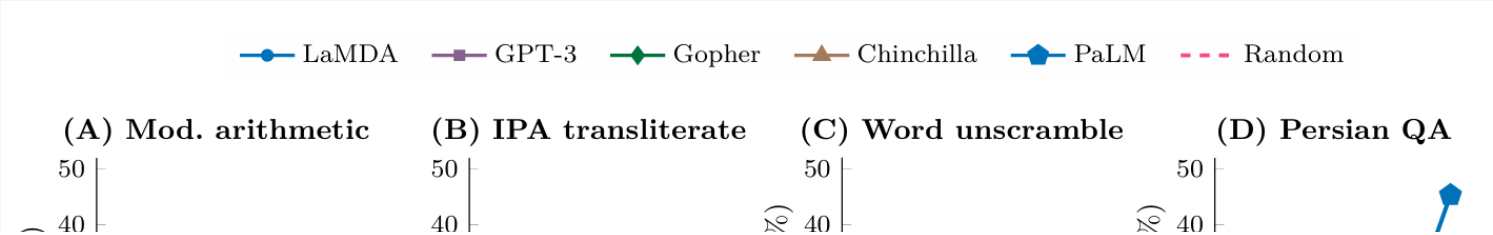
Chain-of-Thought

Problem: "Roger has 5 balls. He buys 2 cans of 3 balls each. How many does he have?"

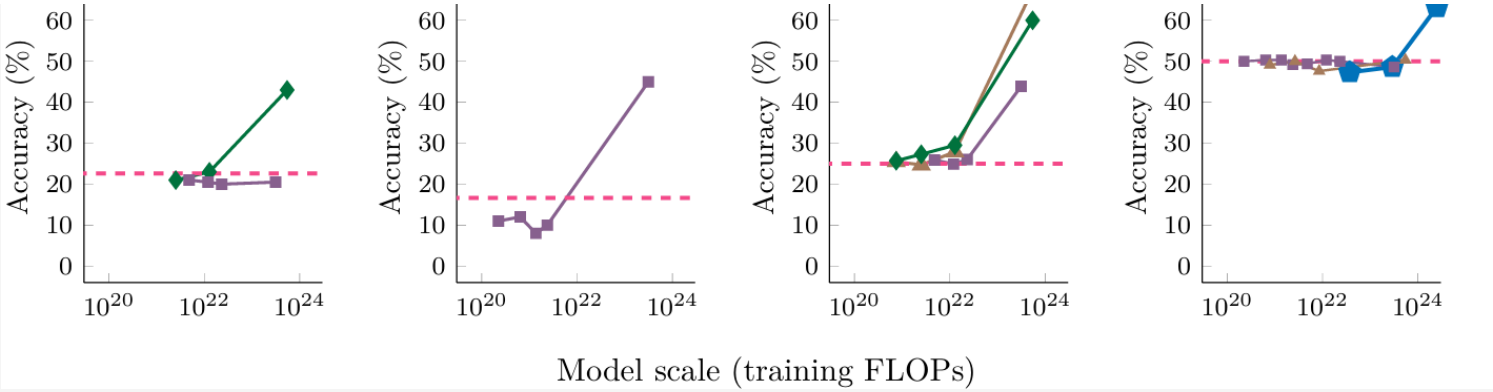
Reasoning: "5 + (2×3) = 11"

Answer: "11 balls"

Scaling Law



Kaplan et al. (OpenAI, 2020)
As you scale model size, training data, and compute, language-model loss typically improves in a predictable power-law way, implying an optimal balance between parameters and data for a given compute budget.



Wei et al. (2022)

Scaling

Models

View all →

Name	Size	Context	Input
ministral-3:latest	6.0GB	256K	Text, Image
ministral-3:3b	3.0GB	256K	Text, Image
ministral-3:8b latest	6.0GB	256K	Text, Image
ministral-3:14b	9.1GB	256K	Text, Image
ministral-3:3b-cloud	-	256K	Text, Image
ministral-3:8b-cloud	-	256K	Text, Image
ministral-3:14b-cloud	-	256K	Text, Image

LLM Application Trends: Chatbot → Reasoning → Agents

2023 ChatGPT Popularizes Conversational AI

ChatGPT demonstrates that LLMs can engage in human-like dialogue. **Main use case: Q&A, brainstorming, content drafting.** The interface is chat-first, and expectations are reset around AI capability.

2024 Multimodal + Real-Time Interaction https://aistudio.google.com/live

Models gain vision, audio, and web browsing. **Main use case: Multimodal understanding, live information retrieval.** The interaction becomes richer, but still primarily conversational.

2025 Reasoning Models + Agents

LLMs shift from "answering" to getting tasks done. Reasoning models (o3, Claude 3.7) can plan, use tools, and execute multi-step workflows.

Tool use | Planning | Execution | Automation

Timeline Takeaways

01

Tokens Define the Budget

Token economics drive cost, latency, and capability. Understanding tokens is essential for reasoning about LLM systems.

02

Transformers Define the Cost Curve

The $O(n^2)$ scaling of attention creates the fundamental trade-off between context length and cost.

03

Answers to Executions

The paradigm shift from answers to executions changes everything about risk and deployment.

Hallucination

Action Error

Over-agreeing

Data Privacy

02 Goals

Reality Check: Where Are We Now ?

1

Q&A / Summarization

Basic question-answering, document summarization. **Entry-level usage.**

2

Repeatable Workflows

Templates, batch processing, standardized prompts. **Process-oriented usage.**

3

Tool Calling (Function Calling)

LLM calls external APIs, databases, calculators. **Connected capabilities.**

4

Agents + Review (Human-in-the-Loop)

Autonomous task execution with human oversight. **Collaborative automation.**

5

Automation + Operations (Production)

Enterprise-grade automation that is **observable, auditable, and rollback-ready.** Not "hands-off AI" but controllable systems.

What Employers Want (Not Just "Prompting")

01

Problem Framing

Can you break down complex problems into LLM-solvable pieces? Employers want people who can architect solutions, not just write prompts.

Example: "Automate customer support" → routing → retrieval → response generation → escalation logic

02

Verification and Accountability

Can you build systems that catch errors before they reach production? Employers need people who understand LLM limitations and design safeguards.

Example: Testing frameworks, human review processes, monitoring dashboards

03

Delivery Under Constraints

Can you deliver reliable solutions within cost, time, and risk constraints? This is the difference between demos and production systems.

Example: Building under token budgets, latency requirements, and safety constraints